

TD7: Ingeniería de Datos

Arquitectura Lambda.
Master Dataset.

Arquitectura Lambda - Almacenamiento

Almacenamiento



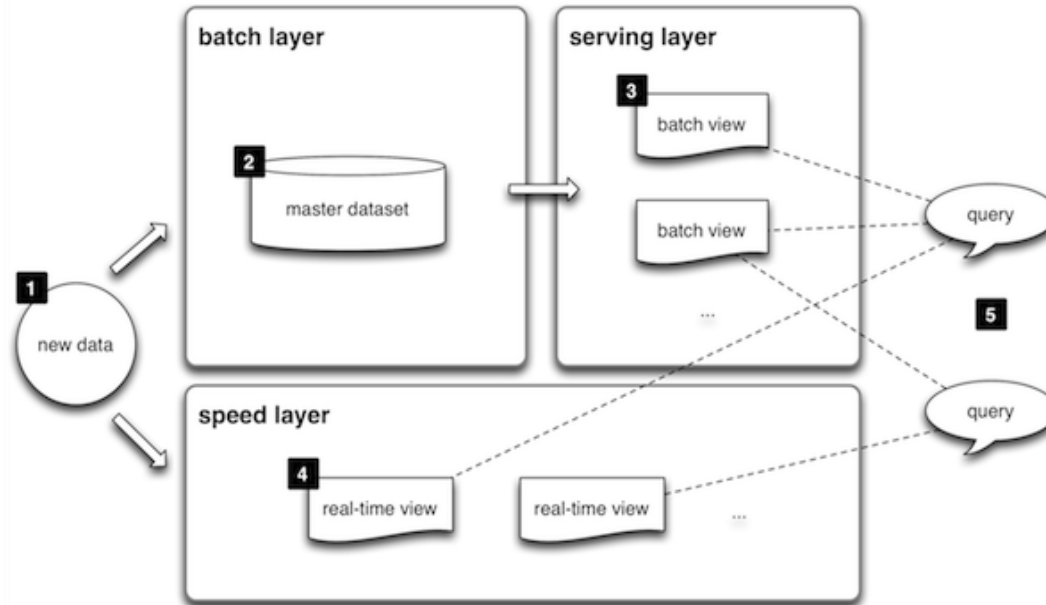
— — —

Muy grandes volúmenes de datos: **eventos**

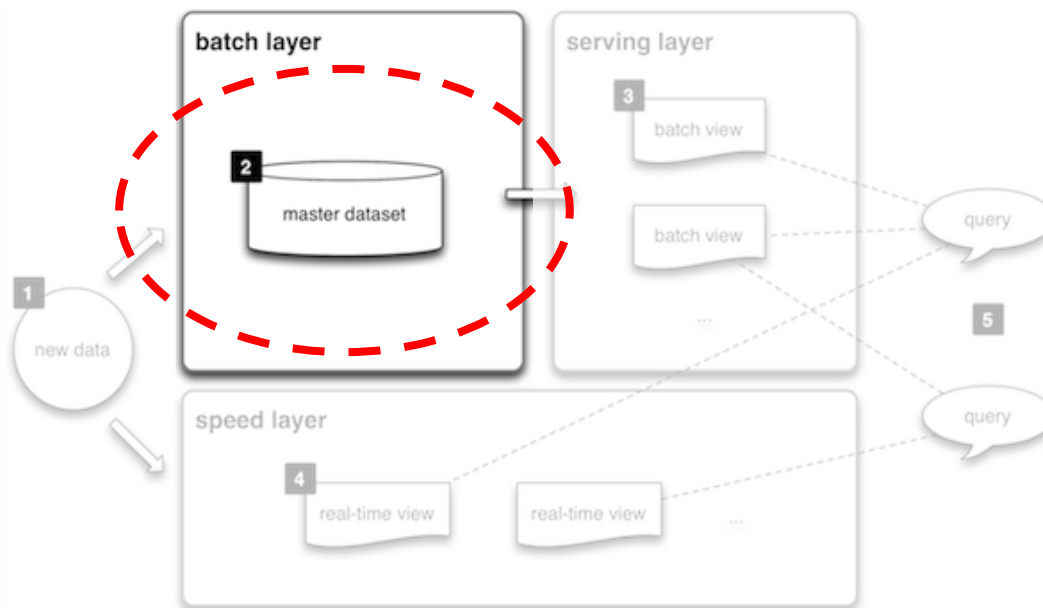
¿En qué forma nos conviene almacenarlos?

- ¿Hasta dónde nos importa su **estructura**?
 - Data Lake vs Data Warehousing
- ¿Qué **uso** les vamos a dar a estos datos?
 - Fully incremental vs Event sourcing
- ¿Qué tipo de **infraestructura** es más adecuada para soportarlos?

Almacenamiento - Master Dataset



Almacenamiento - Master Dataset



Almacenamiento - Master Dataset



Master Dataset

- Es el core de la arquitectura lambda.
 - Source of truth
- Si se perdieran **todos** los datasets de la serving layer y de la speed layer, se podría recuperar todo desde aquí.
 - Esto porque las batch views son producidas por la serving layer como función del master dataset y como la speed layer se basa en la data más reciente, todo se podría reconstruir en pocas horas.

Almacenamiento - Master Dataset



Master Dataset

- **Es la única parte** de la Arquitectura Lambda que debe estar absolutamente a resguardo.
- Recordar que: máquinas sobrecargadas, discos que fallan, cortes de energía, errores humanos son riesgos intrínsecos que inevitablemente ocurrirán.

Las propiedades de los datos

— — —



Supongamos que estamos diseñando la próxima gran red social.

Un nuevo usuario (Tom) se une a la red y empieza a invitar a sus amigos y familia.

¿Qué tipo de información se debería almacenar acerca de las conexiones de Tom?

- **La secuencia de eventos** de las amistades y enemistades de Tom.
- **La lista de amigos** actual de Tom.
- **El número de amigos** actual de Tom.

Las propiedades de los datos



17/may Add Alice
18/may Add Jerry
19/may Add Charlie
20/may Remove Jerry
21/may Add David
22/may Remove Charlie
23/may Add Bob
.
.
.
.



Lista actual
de amigos

Alice
Bob
David



Cantidad
actual de
amigos

3

Almacenamiento - Master Dataset

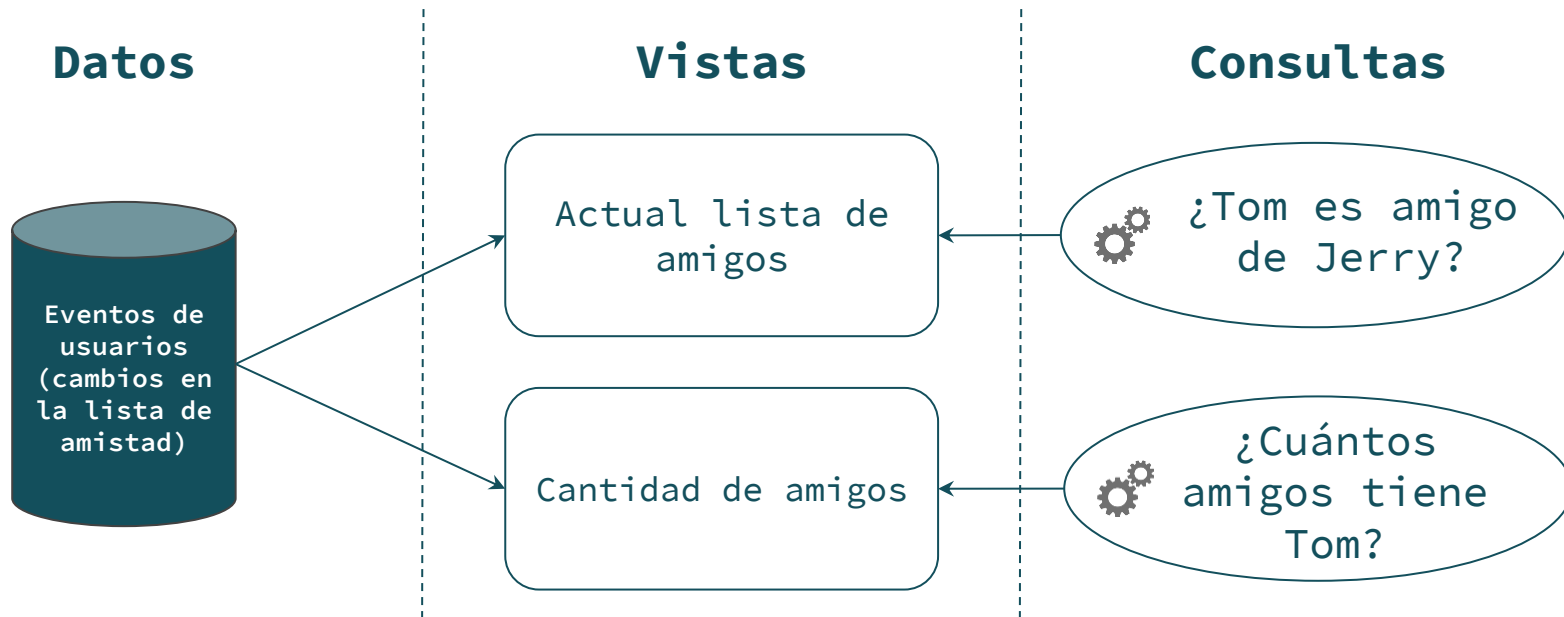


— — —

La idea de **dependencia de la información** nos lleva a estas definiciones:

- **Datos (*data*):** “axiomas” de información que no pueden ser derivados de otros elementos
- **Información:** el conjunto total de conocimiento (incluyendo los datos) relevante al sistema
- **Consultas (*queries*):** preguntas que se le pueden hacer a la información con un dominio en mente
- **Vistas (*views*):** información derivada de los datos base, generalmente orientada a consultas

Almacenamiento - Master Dataset



Propiedades (no funcionales) de los **datos**

Las propiedades de los datos



Tres propiedades claves de los datos:

- **Crudeza**
- **Inmutabilidad**
- **Perpetuidad**

Comprender los fundamentos de Big Data es asimilar estos conceptos.

Comparar con un ambiente relacional donde constantemente se actualiza y resume información para reflejar el estado actual. Los términos “inmutabilidad” y “perpetuidad” no tienen sentido.

Almacenamiento - Master Dataset



Crudeza (*rawness*)

Los datos son tan crudos (no procesados) como se pueda.

- Capacidad de derivar más información (incluso la que todavía no se sabe que se busca) cuanto más crudos son:

Ana hizo
fav a 3
canciones

Ana hizo
fav a las
canciones
#829, #287,
#7110

Ana hizo:

- fav a #829 el 04/10/2023
- fav a #287 el 07/10/2023
- fav a #7110 el 12/11/2023

Ana hizo:

- fav a #829 el 04/10/2023 a las 03:42 pm
- fav a #287 el 07/10/2023 a las 7:12 pm
- fav a #7110 el 12/11/2023 a la 1:05 am

Ana hizo:

- fav a #829 el 04/10/2023 a las 03:42 pm en la web
- fav a #287 el 07/10/2023 a las 7:12 pm en desktop
- fav a #7110 el 12/11/2023 a la 1:05 am en mobile



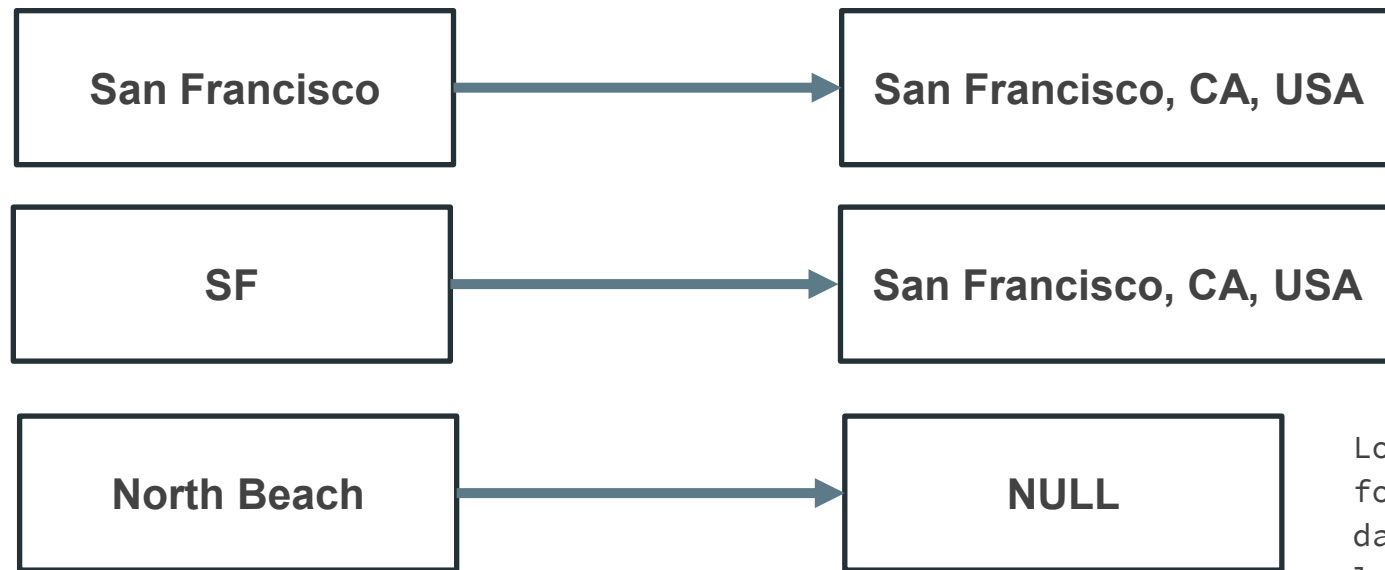
Almacenamiento - Master Dataset



Crudeza (*rawness*)

- Los datos crudos suelen ser **no estructurados antes que normalizados**.
- Cuando se decide qué datos crudos almacenar, hay una fina distinción entre simplemente *parsear* los datos o hacer una *normalización semántica*.
- *Normalización o estandarización semántica* es el proceso de reorganizar información con formato libre en una forma estructurada de data.

Almacenamiento - Master Dataset



Los algoritmos y formas de procesar los datos pueden cambiar a lo largo del tiempo sin que esto sea un problema

Almacenamiento - Master Dataset



Crudeza (*rawness*)

- La **normalización semántica** implica eliminar cualidades del dato que pueden importar en el futuro
- Una arquitectura más cercana a la del **Data Lake** es más consistente con estas ideas

Los algoritmos y formas de procesar los datos pueden cambiar

Almacenamiento - Master Dataset



Inmutabilidad

- Es un concepto extraño pensado desde una base relacional donde el *update* es una de las operaciones fundamentales.
 - No hay updates ni deletes, sólo se agrega data.
 - Puede haber (pocos) casos especiales donde se borra información.
- Hace al sistema **tolerante a las fallas humanas**.
 - Si ocurre, se puede recomputar el master dataset.
- Simplifica la carga de datos
 - Estructuras *append-only* con archivos planos.

Almacenamiento - Master Dataset



— — —

Información del usuario					
id	nombre	edad	género	empleador	ubicación
1	Alice	25	Female	Apple	Atlanta, GA
2	Bob	36	Male	SAS	Chicago, IL
3	Tom	28	Male	Google	San Francisco, CA
4	Charlie	25	Male	Microsoft	Washington, DC

Almacenamiento - Master Dataset

nombre del usuario		
id	nombre	timestamp
1	Alice	2012/03/29 08:12:24
2	Bob	2012/03/12 09:12:24
3	Tom	2012/05/29 08:30:24
4	Charlie	2014/04/29 11:12:24

edad del usuario		
id	edad	timestamp
1	25	2012/03/29 08:12:24
2	36	2012/03/12 09:12:24
3	28	2012/05/29 08:30:24
4	25	2014/04/29 11:12:24

ubicación del usuario		
id	ubicación	timestamp
1	Atlanta, GA	2012/03/29 08:12:24
2	Chicago, IL	2012/03/12 09:12:24
3	San Francisco, CA	2012/04/04 18:31:24
4	Washington, DC	2014/04/29 11:12:24
3	Los Angeles, CA	2012/06/17 20:09:48

Almacenamiento - Master Dataset



Inmutabilidad

- La ubicación actual de Tom ahora requiere una query: buscar todas las ubicaciones y quedarse con el timestamp más reciente.
- Sólo se registra la información **que cambió**.
 - Cada nuevo hecho de la realidad requiere un nuevo dato
- Garantizar la inmutabilidad trae muchas **ventajas**, en particular asegurar la tolerancia a fallas humanas.

Almacenamiento - Master Dataset



Perpetuidad

- Los datos (facts) son verdaderos **para siempre**
 - La inmutabilidad no tendría sentido sin la perpetuidad.
- Etiquetar cada porción de información con timestamps **es la manera de hacer que los datos sean eternamente verdaderos.**

Almacenamiento - Master Dataset



Perpetuidad

- Dos casos especiales en los que se puede considerar el borrado (y que no son incompatibles con que la data sea eternamente verdad)
 - **Garbage collection** borrar datos con poco valor, para controlar el crecimiento del master dataset. Por ejemplo, una regla que mantenga una ubicación por año por persona, en lugar de toda la historia.
 - **Compliance** con regulaciones gubernamentales o políticas de una compañía (p.ej. GDPR (General Data Protection Regulation), el derecho a ser olvidado)

Almacenamiento - Modelo basado en hechos

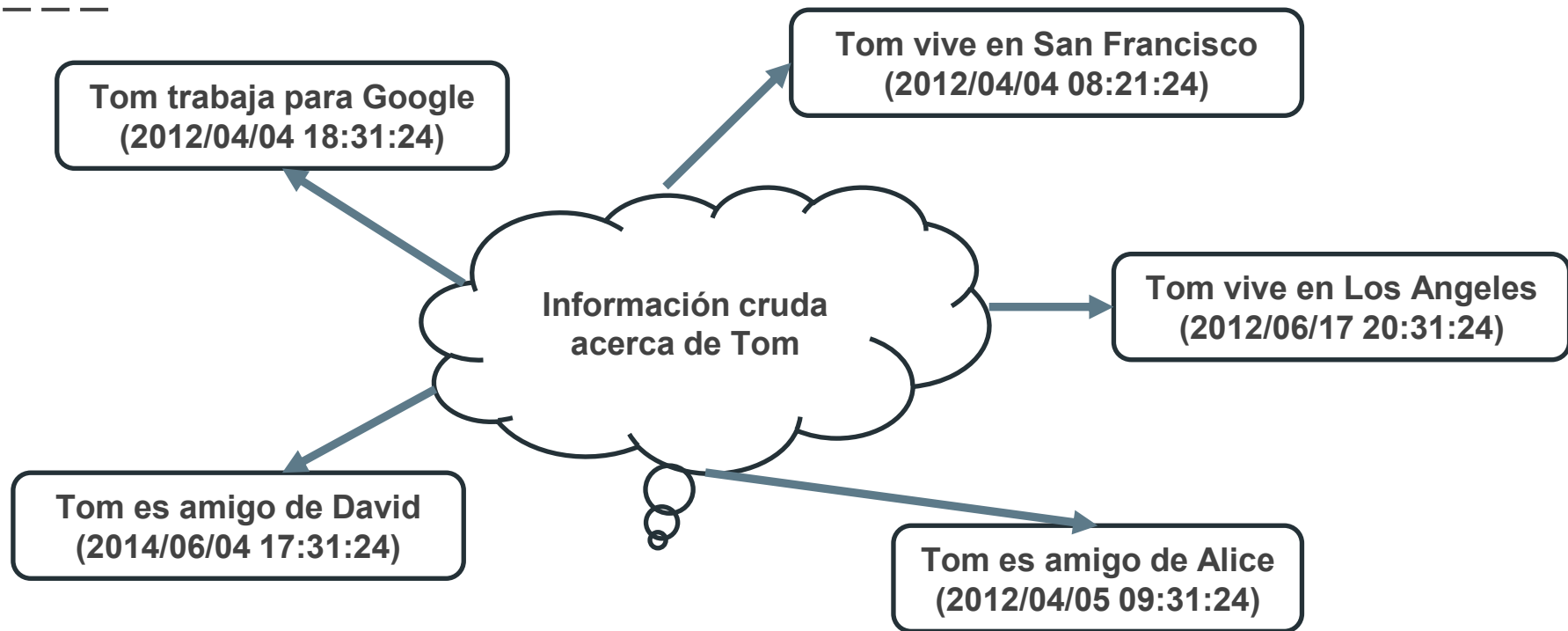


— — —

Un posible acercamiento al almacenamiento de datos (en tanto hechos que son verdad por sí mismos) consiste en definir una estructura específica para cada tipo de dato: el **fact-based model**

Se deconstruye la data en unidades fundamentales (o atómicas) llamadas **facts**.

Almacenamiento - Modelo basado en hechos



Almacenamiento - Modelo basado en hechos



— — —

- Se diseña un **esquema** por cada tipo de evento a almacenar
- Las **distintas partes** relevantes del dato se codifican con su tipo propio
- Todos los eventos tienen un campo de **timestamp** (todo fact tiene un punto específico en el tiempo)
- Se puede agregar un campo extra con un valor generado al azar (un **nonce**) para diferenciar eventos aparentemente idénticos

Almacenamiento - Modelo basado en hechos



Si se quiere almacenar datos acerca del tráfico de la nueva red social:

```
struct PageView:
    DateTime timestamp
    String url
    String ip_address
    Long nonce
```

Almacenamiento - Modelo basado en hechos



— — —

En resumen, esta técnica permite:

- Almacenar los datos como **hechos atómicos**
- Mantener estos hechos **perpetuamente verdaderos** al incluir el timestamp
- Evitar potenciales **duplicados** (dos eventos de iguales características que ocurren al mismo tiempo)

Pero ¿cómo podemos almacenarlos físicamente?

Master dataset

Almacenamiento - Master Dataset



— — —

Uso del master dataset. Típicamente será muy grande para ocupar un único server, así que hay que decidir cómo distribuir la data en múltiples máquinas. La forma en que se almacene el master dataset impactará en cómo se consume.

- De qué formas vamos a querer consumirlo
- Qué problemas tenemos que resolver
- Qué requerimientos no funcionales implica

Almacenamiento - Master Dataset



— — —
Escribir **una vez**, leer **muchas veces** (write once, read many)

- **Write once:** el Master Dataset es inmutable
 - **Grandes volúmenes** de datos en constante **crecimiento**
- **Read many:** casos de uso como ML, generación de batch views, etc
 - En general lecturas **en bulk** y no del tipo **random access**
 - Desde **múltiples procesadores**

La arquitectura e infraestructura de almacenamiento deberían estar optimizadas para estas condiciones

Almacenamiento - Master Dataset



— — —

La **batch layer** es también responsable de computar funciones en el dataset para producir las batch views.

- Esto significa que necesita ser eficiente en **leer mucha data de una vez.**

A continuación, una lista de requerimientos para el almacén de datos:

Almacenamiento - Master Dataset



Operación	Requerimiento	
Escritura	Agregado (append) eficiente de los nuevos datos	Sólo hay escrituras de nuevos eventos/facts
	Almacenamiento escalable	Se almacena el dataset completo por lo que la capacidad debe poder crecer en el tiempo sin mayores dificultades.
Lectura	Procesamiento paralelo	El tipo de cómputo necesario para construir la <i>Batch Layer</i> implica muchas veces recorrer todo el master dataset y es deseable poder paralelizarlo

Almacenamiento - Master Dataset



¿Qué infraestructura de almacenamiento distribuido se adapta mejor como **batch layer storage**?

Key/value stores: la más común de las bases de datos distribuidas

Key Value Store

En el tipo de almacén Key Value, se utiliza una tabla hash en la que una clave única apunta a un elemento.

Las claves pueden ser organizadas por grupos clave lógicos, requiriendo solamente estas claves para ser únicas dentro de su propio grupo. Esto permite tener claves idénticas en diferentes grupos lógicos. La siguiente tabla muestra un ejemplo de un almacén de valores clave, en el que la clave es el nombre de la ciudad y el valor es la dirección de Ulster University en esa ciudad.

Key	Value
"Belfast"	{"University of Ulster, Belfast campus, York Street, Belfast, BT15 1ED"}
"Coleraine"	{"University of Ulster, Coleraine campus, Cromore Road, Co. Londonderry, BT52 1SA"}

Almacenamiento - Master Dataset



Key/value stores:

- Se puede pensar como un gigante hashmap persistente distribuido en muchas máquinas.
- Tiene muchas cosas que no son necesarias para el batch layer storage

Pero van a ser una buena herramienta para otras capas de la arquitectura

Almacenamiento - Master Dataset



File system distribuido:

- Los archivos son secuencias de bytes, y la forma más eficiente de consumirlos es escanando completamente el archivo.
- Los archivos se almacenan secuencialmente en el disco.
- El problema con un filesystem estandard es que funciona en una única máquina.
- La tecnología similar llamada *distributed filesystem* es muy parecida al estandard sólo que desparrama el almacenamiento en un cluster de computadoras.

Almacenamiento - Master Dataset



File system distribuido:

- Diseñados con tolerancia a fallos: si se pierde una máquina todos los archivos y datos serán accesibles.
- Pero las operaciones que se pueden hacer en un filesystem distribuido son más limitadas que las que se pueden hacer en uno estandard.
- Por ejemplo, no se puede escribir en el medio de un archivo o modificar un archivo después de su creación.
- Son ineficientes para pequeños archivos (archivos de 64MB están ok).

Almacenamiento - Master Dataset



File system distribuido:

- Software, mecanismo y estructura para administrar el almacenamiento en cualquier soporte (p.ej. discos).
- Organiza la información en **archivos** (files) y **carpetas** (directories) y se encarga de administrar **permisos** y **optimizar** el uso del soporte para lectura y escritura.
- Utiliza **metadata** de los distintos objetos que se almacena en el mismo soporte.

Almacenamiento - Master Dataset



File system:

- Orientados a **lectura secuencial**
 - No optimizan para lecturas random
- Fácilmente **comprimibles**
- Sistemas robustos de **seguridad**
- Es simple enforzar **inmutabilidad**
- Organización jerárquica de **directorios**

Se ajustan bien a algunos de los requerimientos del Master Dataset.

Formatos de serialización

Formatos de almacenamiento



— — —

Pero ¿Cómo usar el artefacto **archivo** para almacenar los datos?
Algunas opciones simples:

- **Excel**

- Es un formato “estándar”, relativamente compacto
- Pero es un formato propietario y es costoso de leer y escribir

- **CSV: comma separated values**

- Es rápido de leer y escribir
- Pero es muy poco compacto y conlleva problemas como el manejo de caracteres especiales (la coma: “,”)

Formatos de almacenamiento



— — —

En la práctica se requiere una solución más efectiva

Hay **formatos de serialización** que generan archivos binarios y permiten:

- ❖ Definir **esquemas**
 - Para producir mensajes válidos
 - Para validar mensajes
 - Para interpretar mensajes
- ❖ Optimizar el uso de **espacio**
- ❖ Optimizar su **lectura**

Ejemplo

— — —



Python

```
persona = {  
  "nombre": "Ana",  
  "edad": 30  
}
```

JSON

```
{"nombre": "Ana", "edad": 30}
```

Serialización binaria

```
03 41 6E 61 1E
```

Formatos de almacenamiento



— — —

Algunas opciones no tan simples:

❖ Protocol Buffers

- Desarrollado por Google en 2001 y lanzado como open-source en 2008.

❖ Thrift

- Desarrollado y lanzado como open-source por Facebook en 2007.
- Es el formato que usan Marz y Warren en el texto.

Ambos son formatos **binarios** que permiten la comunicación entre procesos que pueden estar implementados en tecnologías diferentes.

Formatos de almacenamiento



— — —

Thrift es un proyecto ampliamente utilizado que se originó en Facebook.

❖ Estructuras

El siguiente código muestra cómo definir una estructura utilizando el lenguaje de definición de interfaz Thrift (IDL). Definir una estructura es como definir una clase en un lenguaje orientado a objetos: especificas todos los datos que contiene el objeto. La diferencia es que una estructura Thrift solo contiene datos y no especifica ningún comportamiento adicional para el objeto. Los campos en una estructura pueden ser:

- Tipos primitivos como cadenas, enteros, largos y dobles. En Thrift IDL, estos se denominan cadena, i32, i64 y doble, respectivamente.
- Colecciones de otros tipos (listas, mapas y conjuntos)

Formatos de almacenamiento

— — —



```
struct Person {  
    1: string twitter_username;  
  
    2: string full_name;  
  
    3: list<string> interests;  
}
```

Formatos de almacenamiento

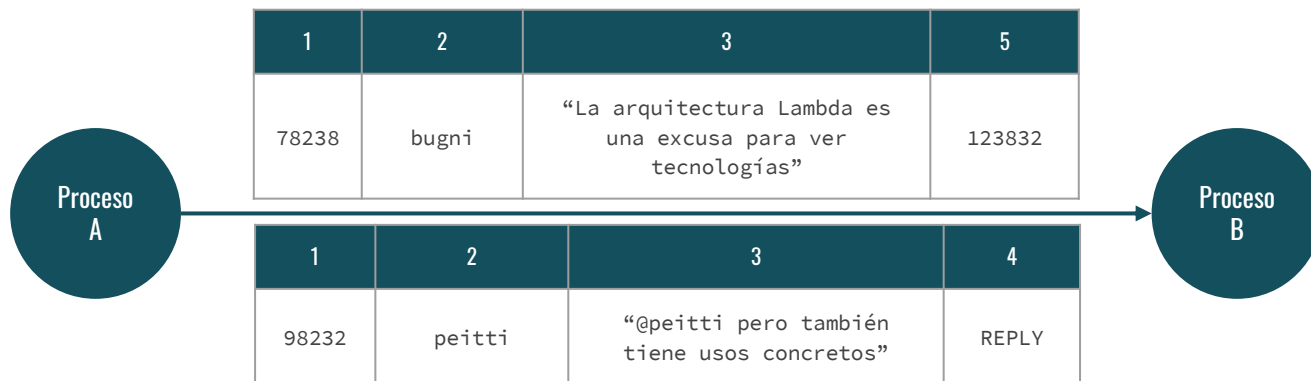


```
struct Tweet {  
    1: required i32 userId;  
    2: required string userName;  
    3: required string text;  
    4: optional TweetType type = TweetType.TWEET;  
    5: optional i32 advertiseId;  
}
```

```
enum TweetType {  
    TWEET,  
    RETWEET,  
    REPLY,  
    QUOTE,  
}
```

```
struct Tweet {  
    1: required i32 userId;  
    2: required string userName;  
    3: required string text;  
    4: optional TweetType type = TweetType.TWEET;  
}
```

```
enum TweetType {  
    TWEET,  
    RETWEET,  
    REPLY,  
}
```



Formatos de almacenamiento



— — —

Los esquemas pueden evolucionar. Ésta es una propiedad crucial porque los requerimientos del negocio pueden cambiar.

- ❖ **Los campos pueden ser renombrados**

Esto porque la forma *serializada* de un objeto se identifica por un ID del campo, no por su nombre.

- ❖ **Un campo puede ser removido pero no puede reusarse su ID**

- ❖ **Sólo campos opcionales pueden agregarse a una estructura existente**

Formatos de almacenamiento



Limitaciones

- ❖ Sólo se chequea que los campos requeridos estén presentes y sean del tipo esperado.
- ❖ No se pueden chequear propiedades como “la edad debería ser no negativa” o “timestamps no deberían estar en el futuro”.
- ❖ La forma correcta de pensar un esquema es como una función que toma una porción de datos y devuelve si es válida o no.

Formatos de almacenamiento



Hay formatos en los que el esquema se almacena junto con el mensaje:

1	2	3	_schema
78238	chiapella	“La arquitectura Lambda es una excusa para ver tecnologías”	1: i32 userId 2: string userName 3: string text 4: enum type { TWEET, RETWEET, REPLY } default TWEET

1	2	3	4	5	_schema
98232	piccoli	“@piccoli pero también tiene usos concretos”	REPLY	30/10/2019	1: i32 userId 2: string userName 3: string text 4: enum type { TWEET, RETWEET, REPLY } default TWEET 5: datetime sentAt

Cierre



Bibliografía

 **“Big data: Principles and best practices of scalable realtime data systems”**, N. **Marz**, J. Warren, 1st edition, 2015.
 Capítulos 2, 3, 4 y 5